

WAF bypasses via 0days

WAF bypasses via 0days - terjanq, @terjanq, Medium.

- Published: 2022-09-23
- Original: <<https://terjanq.medium.com/waf-bypasses-via-0days-d4ef1f212ec>>
- Preserved from: <https://terjanq.medium.com/waf-bypasses-via-0days-d4ef1f212ec> (stored) on 2026-08-09
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Bug Bounty

Hacking

WAF bypasses via 0days

based on findings from a live hacking event

[[image: terjanq]](https://terjanq.medium.com/?source=post_page---byline--d4ef1f212ec-----)

[terjanq](#)

In May, I participated in [1337up0522](#) from Intigriti which was about hacking OWASP ModSecurity Core Rule Set (CRS). I've got 13 findings accepted including 3 exceptional, 2 critical, and 8 high severity vulnerabilities.

In this article, I will showcase a couple of interesting findings.

Content-type confusion

I've discovered that two rules in [ModSecurity/modsecurity.conf-recommended](#) can lead to content-type confusion between WAF and a backend server.

XML request body processor

The regular expression in `SecRule REQUEST_HEADERS:Content-Type "(?:application(?:/soap\+|/)|text/)xml"` will use XML request body processor if it matches the received content-type. However, it misses

matching to the beginning of the string which makes `Content-Type: application/x-www-form-urlencoded;boundary="application/xml"` be interpreted as XML on the WAF side. Because ModSecurity will completely ignore comments in XML the following request body will result in a complete WAF bypass:

```
<a><!-- &email='or'1&password=admin123&'or'1 --></a>
```

JSON request body processor

A similar issue existed in the rule `SecRule REQUEST_HEADERS:Content-Type "application/json"` but the exploitation was trickier. I could trick the WAF to think that `Content-Type: application/x-www-form-urlencoded;boundary="application/json"` is a JSON body but most payloads would still be blocked as I didn't find an easy way of hiding the payload. Initially, because later on I managed to escalate the finding with the following request body:

```
"&email='or 1--"
```

Why does it work? The reason is that even a normal request like:

```
POST /juiceshop/rest/user/login HTTP/1.1
Host: sandbox.coreruleset.org
Content-Length: 16
Content-Type: application/json
Connection: close

"&email='or 1--"
```

wasn't blocked by WAF as it didn't have any parameters — just a single variable with the body contents. And that scenario wasn't covered by CRS rules. So I could trick the WAF into thinking that the request is JSON but smuggle data as `application/x-www-form-urlencoded`.

Multipart parsing 0day

I knew about `multipart/form-data` content-type but never bothered learning how it works in detail. Having a live hacking event motivated me enough to dive deeply into the format and try to discover some nuances. Multipart is much simpler than I initially thought and follows a simple convention.

- Define boundary in content-type, e.g. `Content-Type: multipart/form-data; boundary=xx`
- Use `\r\n` as line terminator
- Start the request part with the boundary prefixed with `--` and end it with the line terminator
- Include headers such as `Content-disposition` in consecutive lines separated by the line terminator

- Separate headers and body with line terminator
- End the request part with either a new part or the end of the request which is the boundary prefixed and suffixed with `--`.

That's what a simple multipart request might look like:

```
POST /juiceshop/rest/user/login HTTP/1.1
Host: sandbox.coreruleset.org
Content-Length: 130
Content-Type: multipart/form-data; boundary=xx
Connection: close

--xx
Content-disposition: form-data; name="username"terjanq
--xx
Content-disposition: form-data; name="password"

password
--XX--
```

Empty body

Immediately after learning the format, I spotted an imperfection in the algorithm. What happens if a part does not have any body??

```
--xx
Content-disposition: form-data; name="username"

--xx
Content-disposition: form-data; name="jj"; filename="ccc"

'or'1
--xx
Content-disposition: form-data; name="password"

test123
--XX--
```

Notice that the first part does not contain a body. Depending on how the parser is implemented it might either think that `--xx` is the body, or it might realize that a new part started and consider that part empty. Unfortunately for ModSecurity, they considered the request as the latter which resulted in printing three variables:

- `username=`
- `FILE: jj='or'1`

- `password=test123`

Because files are ignored by CRS, it will find nothing suspicious in the request. However, some backends will disagree with ModSecurity and will understand the request as:

- `username=--xx\r\nContent-disposition: form-data; name="jj"; filename="ccc"\r\n'or'1`
- `password=test123`

This for example caused all PHP backends to become bypassable.

Body terminated with a new line

Another issue is strictly related to ModSecurity. They incorrectly considered a single new line terminator `\n` as `\r\n` after the body.

Hence, the body:

```
--boundary\r\n
Content-disposition: form-data; name="parameter"\r\n
\r\n
body\r\n
--boundary\r\n
Content-disposition: form-data; name="parameter2"\r\n
\r\n
non-empty value
--boundary--
```

was translated by ModSecurity to two variables:

- `parameter=body`
- `parameter2=non-empty value`

Whereas almost all backends will understand it as:

```
parameter=body\r\n--boundary\r\nContent-disposition: form-data; name="parameter2"\r\n\r\nnon-empty value
```

Exploitation is almost the same as in the previous section with a slight advantage that the attacker fully controls the prefix of the request.

Charset confusion (CVE-2022-39955)

This issue was considered as high severity even though it allowed to completely bypass rules when the backend understands `utf-7` encoding. I discovered that CRS only looks at the first `charset=` and blocks it if it's not `utf-8`. With the content-type `Content-Type: application/json;charset=utf-8;charset=utf-7` I bypassed the rule and made express application decode the request as `utf-7`. UTF-7 is a very interesting format that lets you encode any characters as base64 therefore the body `+ACcAbwByACcAMQAnAC0ALQ-` is understood as `'or'1` — by the backend whereas the WAF finds nothing suspicious in the request.

Closing thoughts

As I enjoyed finding all those bugs in my after-work free time, the event itself was very stressful. It took weeks for the reports to be resolved with me spending lots of time explaining the impact. I wish I could spend all that time discovering more issues. A few of my reports are still not finalized which only adds up to the experience.

With that said, Intigriti did a great job and the idea for the event was awesome. Hacking a non-profit open-source project to improve the overall security of dozens of products relying on it is undoubtedly awesome!

For hackers expecting to see me on the Live Hacking Event in Belgium, unfortunately I needed time to detach and had decided not to participate. Maybe next time!

*All bugs are patched in the newest releases of ****ModSecurity*** and ***CRS**.*

Archived from <https://terjanq.medium.com/waf-bypasses-via-0days-d4ef1f212ec>. Rendered offline from the local Markdown copy.